P. Abhilesh Sai
Section :- 4
Adm no:- 1363.

Assignment - 2

1. Reverse a string using stack

Algorithm

1. start
2. Read string
3. push each character into stack.
4. Pop characters one by one.
5. Print popped characters [Reversed string]
6. stop.

Code:

```
#include<stdio.h>
#include <string.h>
#define MAX 100
 char stack[MAX];
 int top=-1;
//push function
void push (char ch){
    stack[++top]=ch;
}
```

```
// function to delete element

void dequeue () {
    if (front == -1) {
        printf("Queue is empty");
    }
    else
    {
     int data= queue is empty {
     int data= queue [front];
      if (front == rear)
      {
       front=rear = -1;
       // queue becomes empty
      }
      else
      {
        front=(front + 1)% SIZE;
      }
     printf("%d removed from queue \n", data);

    }

}
```

```
// function to see front element
void peek() {
    if (front == -1) {
    printf("Queue is Empty");
    } else {
    printf("front element: %d\n", queue[front]);
    }
}
// function to display queue
void display(){
    if (front == -1) {
    printf("Queue is Empty:\n");
    } else {
    printf("Queue elements:");
    int i = front;
    while (1) {
        printf("%d", queue[i]);
        if (i == rear) break;
        i = (i+1) % SIZE
    }
    printf("\n");
    }
}
```

```
// Main menu
int main()
    int choice, value;
    while (1) {
        printf("1. Enqueue 2. Dequeue 3. Peek 4. Display
                5. Exit");
        printf("Enter your choice:");
        scanf("%d", &choice);
        switch (choice) {
        Case 1: printf("Enter value:)
                scan f("%d", &value);
                enqueue(value)
                break.
        case 2 : dequeue(); break;
        case 3 : peek(); break;
        case 4 : display(); break;
        case 5 : return 0;
        default : print("Invalid choice");

    }

}

}
```

output

1. Enqueue 2. Dequeue 3. Peek 4. Display 5. Exit

Enter your choice: 1

Enter value to insert: 10

10 inserted into queue.

```
//pop function
char pop(){
  return stack[top--];
}
int main(){
   char str[100];
   int i;
 printf("Enter string:");
 scanf("%s",&str);
//push all characters
for (i=0; i<strlen(str); i++){
      push(str(i));
   }
   printf("Reversed string:");
//pop all characters
  while (top!=-1){
   printf("%c", pop());
   }
  return 0;
}
```

output

Enter string : hettler

Reversed string : rettel

Balanced Paratheses

Algorithm:

1. Scan expression
2. Push opening brackets
3. On closing bracket -> check match top.
4. If mismatched -> not Balanced
5. At end -> if stack empty -> Balanced.

Code:

```
#include<stdio.h>
# include< string.h>
# include<ctype.h>
# define MAX 50
  char stack [MAX];
  int top=-1
  void push(char c){ stack [++top]=c;}
  char POP( ){ return stack[top--];}
  void
  int match (char a, char b){
      return (a=='('&&b==')')||(a=='{'&&b=='}')||
              (a=='['&&b==']');
  }
      int check (char *exp){
      for (int i=0;i<strelen (exp); i++){
  // push open
        if (exp[i]=='('||exp[i]=='{'||exp[i]=='[')
        push(exp[i]);
```

```
    else if (exp[i]==')' || exp[i]=='}' || exp[i]==']')
  // mismatch
      if (top==-1 || match(stack[top], exp[i]))
        return 0;
        pop();
      }
    }
    return top==-1; //Balanced if empty.
  }
  int main(){
    char exp[Max];
    printf("Enter expression:");
    scanf("%s", exp);
    if(check(exp) printf("Balanced expression\n");
    else printf("not Balanced Expression\n");
    return 0;
  }
```

Output

Enter expression: (A+B)*(C-D)

Balanced expression

3. Next Greater element

Algorithm

1. start
2. Take array input
3. Transer array.

4. Use stack to track elements

5. for each element

-> Compare with stack top

-> print next greater

6. stop.

Code:

```
#include <stdio.h>
# define MAX 100        //define stack size
  int stack [MAX];
  int top=-1;           // initialize top
  void push (int x) {   //push element into stack
   stack [++top]=x;
  }
  int pop() {
     return stack [top--];
   }
  int main () {
   int arr[ ] = {5, 6, 7, 8, 10};
   push (arr[0]);
   for (i=1; i<n; i++){
   while (top != -1 && stack [top]<arr[i]){
      printf ("%d > %d \n", pop(), arr[i]);

   }
   push (arr[i]);

  }
```

```
while (top != -1) { //(print -1 if no greater element)
  printf("%d -> -1 \n", pop ());
  }
  return 0;
}
```

Output

5 -> 6

6 -> 7

7 -> 8

8 -> 10

10 -> -1

4. printer queue simulation.

Algorithm

1. start
2. Initialization queue
3. Display Menu (Add, Print, Display)
4. Perform operations using FIFO
5. stop.

Code;

```
#include<stdio.h>
#define Max 5          //define queue size.
  int queue [MAX];
  int front = -1, rear = -1; //Initialize front and rear
  //void enqueue (int x){
    if (rear == MAX-1){ //check queue full
      printf ("Queue full \n");
      return;
    }
```

```
void dequeue(){    //remove element
    if (front == -1){
        printf("Queue Empty \n");
        return;
    }
    printf("printed document: %d \n", queue[front++]);
}
void display(){    // display queue

    int i;
    if (front == -1){
        printf("No pending documents \n");
        return;
    }
    printf("pending documents:");
    for (i = front; i <= rear; i++)
        printf(" %d", queue[i]);
    printf("\n");
}
int main(){
    int choice, doc;
    while(1){    //menuloop
        printf("\n 1.Add 2.print 3.Display 4. Exit \n");
        scanf("%d", &choice);
        switch (choice){
        Case 1: printf("Enter id: "); scanf("%d", &doc); enqueue(); break;
        Case 2: dequeue(); break;
        Case 3: display(); break;
        case 4: return 0;    //exist program.
        }
    }
```

Output

1. Add 2. print 3. Display 4. Exit

Enter id : 10

5. Circular Queue Basic operations

Algorithm

1. Initialize front == 1, rear == -1;
2. Enqueue: insert element at queue [rear]
3. Dequeue: remove from front.
4. peek: return queue [front]
5. Display: start from front and print until rear ((i+1) % size)

Code:

```
#include <stdio.h>
#define SIZE 5   // fixed of circular queue
int front = -1, rear = -1;
// function to Insert element
void enqueue (int values) {
    if ((rear+1) % SIZE == front) {
    printf ("Queue is full: \n");
}
else {
    if (front == -1) front = 0; // first element
    rear = (rear+1) % SIZE;
    queue [rear] = value;
    printf("%d inserted into queue: \n", value);
}
}
```